========== Multivariate Regime-Conditional Simulator ==========

Simulated World Project.

Part 1.3: Synthetic Worlds. Factors, Return SImulation

In [101…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KernelDensity
from hmmlearn.hmm import GaussianHMM
from scipy import stats
from sklearn.linear_model import LinearRegression
from statsmodels.stats.diagnostic import acorr_ljungbox
```

Load data

Real world Data.

In [2]:

```
# All features includes 'return', 'fwd_return', 'regime'
data_2 = pd.read_csv("project_DATA/_vet_features_df.csv", index_col="Date", parse_dates=True)
vet_features_df = data_2.copy(deep=True)
print("======== Full set of features. vet_features_df:")
vet_features_df
```

```
======== Full set of features. vet_features_df:
```

Out[2]:

Date	Volume	VIX_indx_close	term_spread	T10Y3M	T10Y2Y_yield_curve	DGS10_dif	T10Y2Y_dif	DTB3_dif	news_sentiment	is_GDP_release_day	...	AD
2000-04-12	1175900000	28.980000	6.40	0.30	1	0.08	0.04	0.00	0.25	0	...	0.
2000-04-13	1032000000	29.400000	6.40	0.29	1	-0.03	-0.03	-0.02	0.23	0	...	0.
2000-04-14	1279700000	33.490002	6.27	0.20	1	-0.09	0.04	0.00	0.22	0	...	0.
2000-04-17	1204700000	28.950001	6.33	0.36	1	0.16	0.10	0.00	0.20	0	...	0.
2000-04-18	1109400000	26.120001	6.33	0.40	1	0.04	0.04	0.00	0.20	0	...	0.
...	...	...	...	...	...	...	...	...	...	...	...	
2025-12-23	3820560000	14.000000	3.48	0.62	0	0.01	-0.03	0.00	-0.02	1	...	0.
2025-12-24	1798270000	13.470000	3.47	0.55	0	-0.03	-0.02	0.04	-0.00	0	...	0.
2025-12-26	2586550000	13.600000	3.46	0.58	0	-0.01	0.00	-0.04	-0.01	0	...	0.
2025-12-29	3541750000	14.200000	3.45	0.55	0	-0.02	-0.01	0.01	0.04	0	...	0.
2025-12-30	3309930000	14.330000	3.45	0.59	0	0.02	0.02	-0.02	0.03	0	...	0.

6468 rows × 68 columns

In [3]:
```
# Factors metrics sorted by pro_score_penalized. Correlated
data_3 = pd.read_csv("project_DATA/_pro_score_df_metrics_factors.csv")
factor_metrics_sorted = data_3.copy(deep=True)
```

In [4]:
```
# Factors-Metrics based on selected factors
factor_metrics = factor_metrics_sorted.T
headers = factor_metrics.iloc[0].values
factor_metrics.columns = headers
factor_metrics = factor_metrics.drop(index='factors')
print("======== factor_metrics:")
factor_metrics
```

```
======== factor_metrics:
```

Out[4]:

	RSI_7_norm	RSI_14_norm	price_vs_EMA20	price_vs_EMA10	price_vs_SMA50	BB_position	high_close_ratio	price_vs_SMA20	ROC_10	close_
pro_score	2.098176	1.948817	1.618108	1.56993	1.361215	1.305819	1.265156	1.166035	1.060929	
robust_score_3	1.868698	1.711771	1.431211	1.496106	1.132731	1.281385	1.311002	1.084484	1.05212	
ic_all	-0.056772	-0.048241	-0.044146	-0.05007	-0.03262	-0.047682	0.059749	-0.040647	-0.041829	-
ic_calm	-0.08233	-0.079805	-0.066322	-0.064675	-0.058681	-0.07143	0.079496	-0.063137	-0.049404	-
ic_stress	-0.049543	-0.041717	-0.044277	-0.050893	-0.03663	-0.035603	0.085627	-0.036647	-0.035603	-
regime_gap	0.032787	0.038087	0.022045	0.013782	0.022052	0.035827	0.006132	0.026491	0.013801	
vol_sensitivity	0.034777	0.041967	0.032597	0.021256	0.036608	0.042795	0.007601	0.037935	0.033933	
stability_metric	1.391535	1.455437	1.378795	1.155797	1.563794	1.161985	0.949948	1.264372	1.08059	
bootstrap_IC_mean	-0.057173	-0.0487	-0.043954	-0.050422	-0.032166	-0.046713	0.060148	-0.040088	-0.041503	-
bootstrap_IC_interval	0.042574	0.041407	0.042344	0.038953	0.044407	0.04236	0.043583	0.046738	0.042626	
ci_contains_zero	False	False	False	False	False	False	False	False	False	
gap_gt_ic	False	False	False	False	False	False	False	False	False	
pro_score_penal	2.098176	1.948817	1.618108	1.56993	1.361215	1.305819	1.265156	1.166035	1.060929	
score_delta	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	

In [5]:
```
factor_metrics.shape
```

Out[5]:
```
(14, 19)
```

In [6]:
```
# Read selected_factors
print("======== Selected Factors:")
selected_factors_list = factor_metrics.columns.to_list()
selected_factors_list
```

```
======== Selected Factors:
```

Out[6]:
```
['RSI_7_norm',
 'RSI_14_norm',
 'price_vs_EMA20',
 'price_vs_EMA10',
 'price_vs_SMA50',
 'BB_position',
 'high_close_ratio',
 'price_vs_SMA20',
 'ROC_10',
 'close_position',
 'body',
 'ROC_20',
 'Low_return',
 'news_sentiment',
 'Open_return',
 'upper_wick',
 'DGS10_dif',
 'kin_f3_cpi_t10y3m',
 'T10Y3M']
```

In [7]:
```
# the Markov transition matrix learned by GaussianHMM on real data
hmm_matrix = np.load('project_DATA/HMM_real_world_transition_matrix.npy')
trans_mat = hmm_matrix.copy()

print("======== HMM_trans_mat:")
print(trans_mat)
print(trans_mat.shape)
```

```
======== HMM_trans_mat:
[[9.79538250e-01 2.04617504e-02 1.06263579e-90]
 [1.80061367e-02 9.72568467e-01 9.42539597e-03]
 [1.87665236e-58 2.16112139e-02 9.78388786e-01]]
(3, 3)
```

In [ ]:
```
# For research perpouse matrix. More aggresive stress regime
#trans_mat_research = [[0.97, 0.025, 0.005], [0.009, 0.97, 0.021], [0.001, 0.029, 0.97]]
```

Real world HMM Regime Statistics

In [8]:
```
# HMM Regime Statistics
def hmm_regime_stat(df, col):
    sub_df = df[[col]].copy(deep=True)
    sub_df['run_id'] = (sub_df[col] != sub_df[col].shift()).cumsum()
    run_stats = sub_df.groupby([col, 'run_id']).size().reset_index(name='length')

    print("Section details:")
    print(run_stats)

    print("\nSummary:")
```

```
    summary = run_stats.groupby(col).agg(
        num_sections=('run_id', 'nunique'),
        total_count=('length', 'sum')
    )
    print(summary)

    return run_stats
```

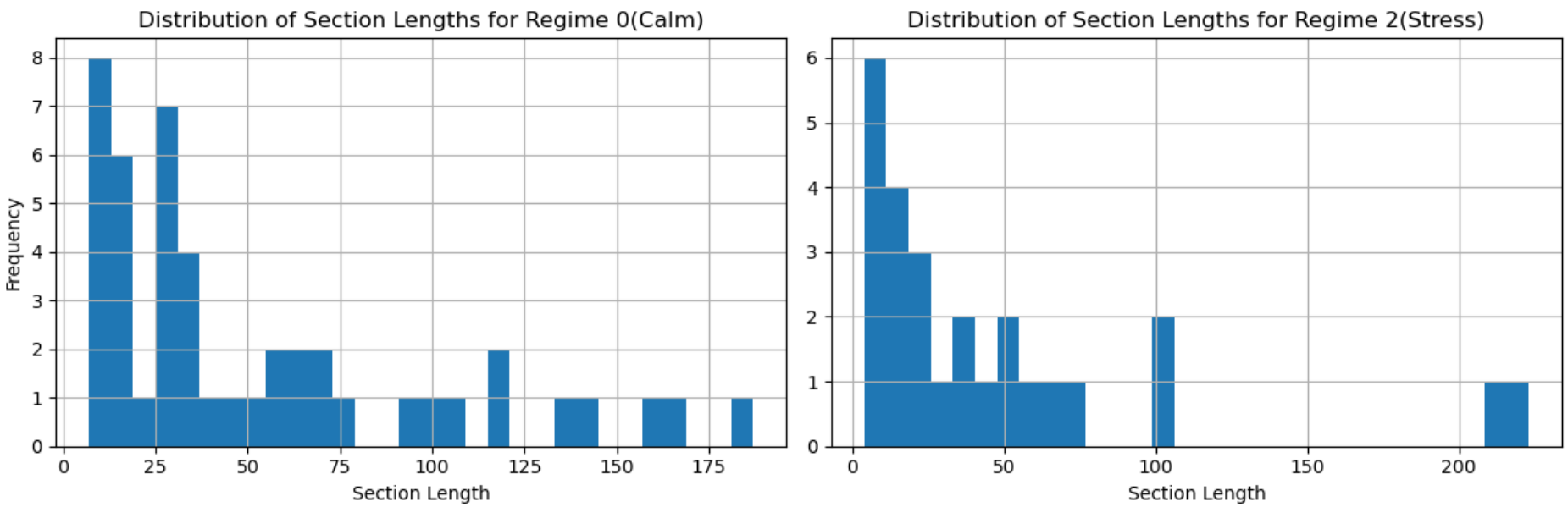
In [9]:
```
# Real Stat
run_stat_real = hmm_regime_stat(vet_features_df, col='hmm_regime')
```

```
Section details:
     hmm_regime  run_id  length
0             0      13      37
1             0      15      17
2             0      17      48
3             0      19     164
4             0      21     116
..          ...     ...     ...
138           2     119       6
139           2     121      21
140           2     123      59
141           2     125      75
142           2     139      31

[143 rows x 3 columns]

Summary:
            num_sections  total_count
hmm_regime
0                     46         2431
1                     71         2773
2                     26         1264
```

In [10]:
```
# Calm(regime 0) vs Stress(regime 2) Distributions Vizualization
def calm_stress_regime_viz(stats_df, col):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Plot for regime 0
    stats_df[stats_df[col] == 0]['length'].hist(ax=axes[0], bins=30)
    axes[0].set_title('Distribution of Section Lengths for Regime 0(Calm)')
    axes[0].set_xlabel('Section Length')
    axes[0].set_ylabel('Frequency')

    # Plot for regime 1
    stats_df[stats_df[col] == 2]['length'].hist(ax=axes[1], bins=30)
    axes[1].set_title('Distribution of Section Lengths for Regime 2(Stress)')
    axes[1].set_xlabel('Section Length')

    # Plot for regime 2
```

```
    plt.tight_layout()
    plt.show()
```

In [11]:
```
# Check the regimes destribution on Real world
calm_stress_regime_viz(run_stat_real, col='hmm_regime')
```



simulate Regime

Simulate the world, Copula path

In [12]:
```
# 1. Simulate regimes. Use the real world HMM probability matrix
def simulate_regimes(T, trans_mat, init_state=0):
    states = [init_state]

    for _ in range(T-1):
        current = states[-1]
        next_state = np.random.choice(
            len(trans_mat),
            p=trans_mat[current]
        )
        states.append(next_state)
        col = ['hmm_regime']
```

```
    return pd.DataFrame(data=np.array(states), columns=col)
```

In [52]:
```
# number of observations
T=len(vet_features_df)
# NEW TRANS_MAT
#trans_mat = trans_mat_research
```

In [ ]:
```
# Simulate regimes
```

In [57]:
```
simulate_regimes_df = simulate_regimes(T, trans_mat, init_state=2)
simulate_regimes_df['hmm_regime'].value_counts(dropna=False)
```

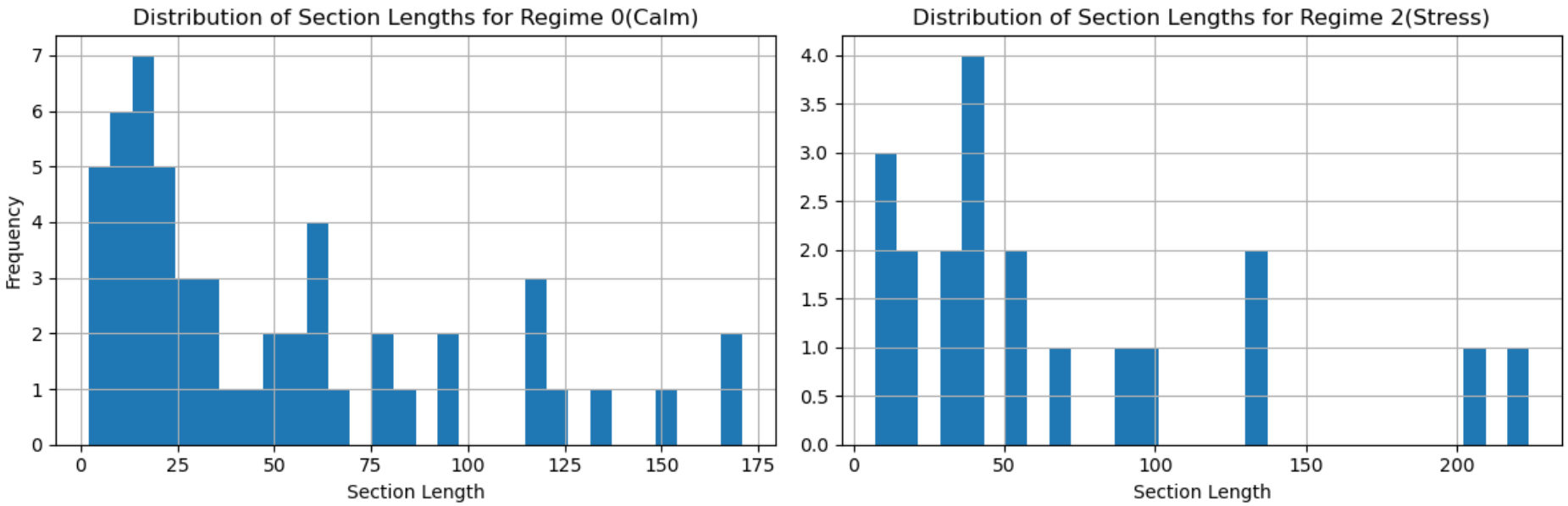
Out[57]:
```
hmm_regime
0    2641
1    2501
2    1326
Name: count, dtype: int64
```

Simulated world HMM Regime Statistics

In [58]:
```
# Simulated Stat
run_stat_sim = hmm_regime_stat(simulate_regimes_df, col='hmm_regime')
```

```
Section details:
     hmm_regime  run_id  length
0             0       3      24
1             0       7      34
2             0      11      80
3             0      13      62
4             0      15      18
..          ...     ...     ...
140           2     119       7
141           2     129      36
142           2     133       7
143           2     141      88
144           2     145       7

[145 rows x 3 columns]

Summary:
             num_sections  total_count
hmm_regime
0                      53         2641
1                      72         2501
2                      20         1326
```

In [59]:
```
# Check the regimes destribution on Simulated world
calm_stress_regime_viz(run_stat_sim, col='hmm_regime')
```



Gaussian Copula Simulator

```
'''
Any joint distribution can be decomposed into two separate pieces –
the marginal distributions of each variable individually,
and a copula that describes the dependence structure between them.
'''
```

Fit Multivariate per regime. Goal: to define the "parameters" of selected factors to simulate them

In [60]:
```
count_regime_0 = (simulate_regimes_df['hmm_regime'] == 0).sum()
count_regime_1 = (simulate_regimes_df['hmm_regime'] == 1).sum()
count_regime_2 = (simulate_regimes_df['hmm_regime'] == 2).sum()
count_regime_0, count_regime_1, count_regime_2
```

Out[60]:
```
(np.int64(2641), np.int64(2501), np.int64(1326))
```

In [61]:
```
# ================ Step 1 – Rank-Gaussian transform
def rank_gauss(x):
    r = stats.rankdata(x)/(len(x)+1)
    return stats.norm.ppf(r)
```

In [62]:
```python
# ============== Step 2 — Fit copula per regime. Learning the Dependence
def fit_copula(df, features, regime_col="hmm_regime"):

    models = {}

    for r in sorted(df[regime_col].unique()):
        sub = df[df[regime_col]==r][features].dropna()

        Z = np.column_stack([
            rank_gauss(sub[c]) for c in features
        ])

        Sigma = np.cov(Z.T)

        models[r] = {
            "cov": Sigma,
            "data": sub
        }

    return models
```

In [63]:
```python
# Get Copula destribution models
models = fit_copula(vet_features_df, selected_factors_list)
```

In [64]:
```python
# Factors sampling simulation function
def sample_copula(models, features, regime, n):
    '''
    Generating Synthetic Data
    '''
    m = models[regime]
    Z = np.random.multivariate_normal(
        np.zeros(len(features)),
        m["cov"],
        size=n
    )
    U = stats.norm.cdf(Z)
    # inverse ECDF
    X = np.zeros_like(U)

    '''
    **Inverse ECDF** — uses the uniform values as quantile lookup indices into the original sorted real-world data.
    So if `U=0.9`, it picks the 90th percentile value from the actual observed distribution of that feature.
    This is what makes it non-parametric.
    '''

    for j,c in enumerate(features):
        sorted_vals = np.sort(m["data"][c])
        idx = (U[:,j]*(len(sorted_vals)-1)).astype(int)
```

```
        X[:,j] = sorted_vals[idx]

    return pd.DataFrame(data=X, columns=features)
```

In [65]:
```
# Generate Factors Sample DF based on regime
copula_factors_sim_0 = sample_copula(models, selected_factors_list, regime=0, n=count_regime_0)
copula_factors_sim_1 = sample_copula(models, selected_factors_list, regime=1, n=count_regime_1)
copula_factors_sim_2 = sample_copula(models, selected_factors_list, regime=2, n=count_regime_2)
# Copy
simulated_F_R0 = copula_factors_sim_0.copy(deep=True)
simulated_F_R1 = copula_factors_sim_1.copy(deep=True)
simulated_F_R2 = copula_factors_sim_2.copy(deep=True)

simulated_F_R0.shape, simulated_F_R1.shape, simulated_F_R2.shape
```

Out[65]: ((2641, 19), (2501, 19), (1326, 19))

Concut simuated_regime_df(regime column only) + simulated_F_R0, simulated_F_R1, simulated_F_R2

row by row based on regime mark 0,1,2

In [66]:
```
# Create a mapping array for each regime
regime_indices = {
    0: np.arange(len(simulated_F_R0)),
    1: np.arange(len(simulated_F_R1)),
    2: np.arange(len(simulated_F_R2))
}

# Create a counter for each regime
regime_positions = {0: 0, 1: 0, 2: 0}
regime_column = 'hmm_regime'

# Pre-allocate list for efficiency
rows_list = []

for regime in simulate_regimes_df[regime_column]:
    regime = int(regime)
    pos = regime_positions[regime]

    # Get the specific row
    if regime == 0:
        row = simulated_F_R0.iloc[[pos]]
    elif regime == 1:
        row = simulated_F_R1.iloc[[pos]]
    else:  # regime == 2
        row = simulated_F_R2.iloc[[pos]]

    rows_list.append(row)
```

```
    regime_positions[regime] += 1

# Combine
df_first_sim = pd.concat(rows_list, ignore_index=True)
df_first_sim['hmm_regime'] = simulate_regimes_df[regime_column].values
```

First Factor's Simulated Set

In [67]:
```
# First simulated set of selected factors + 'hmm_regime'
df_first_sim.shape
```

Out[67]: (6468, 20)

Final Set with Cliped Simulated Factor's

In [68]:
```
# Clip sim_factors
def clip_to_empirical_by_regime(df_sim_sim, df_real_real, factors):
    """
    Clip simulated values to empirical quantiles based on regime.
    Parameters:
    df_sim: DataFrame with simulated data (will be modified)
    df_real: DataFrame with real data containing 'hmm_regime' column
    factors: list of column names to clip
    """
    df_sim = df_sim_sim.copy(deep=True)
    df_real = df_real_real.copy(deep=True)

    # Get unique regimes
    regimes = df_real['hmm_regime'].unique()

    for regime in regimes:
        # Select data for this regime
        mask = df_real['hmm_regime'] == regime
        regime_data = df_real.loc[mask]

        if len(regime_data) == 0:
            continue

     # Set quantiles based on regime
        if regime == 0:
            lo_q, hi_q = 0.01, 0.99
        elif regime == 2:
            lo_q, hi_q = 0.001, 0.999
        else:
            # Default quantiles for other regimes
            lo_q, hi_q = 0.01, 0.99

        # Clip each feature for this regime
```

```
        for col in factors:
            if col in regime_data.columns:
                lo = regime_data[col].quantile(lo_q)
                hi = regime_data[col].quantile(hi_q)

                # Apply clipping only to rows in this regime
                regime_mask_sim = df_sim['hmm_regime'] == regime
                df_sim.loc[regime_mask_sim, col] = df_sim.loc[regime_mask_sim, col].clip(lo, hi)

    return df_sim
```

In [69]:
```
# Simulate clipted_factors, no returns simulated
df_clipped_sim = clip_to_empirical_by_regime(df_first_sim, vet_features_df, selected_factors_list)
```

Check the factors destribution on real world vs simulated

In [70]:
```
# Real world Selected factors destrib
print(f"==================")
print(f"||     Real     ||")
print(f"==================")
vet_features_df[selected_factors_list + ["hmm_regime"] + ['return']].describe()
```

```
==================
||     Real     ||
==================
```

Out[70]:

	RSI_7_norm	RSI_14_norm	price_vs_EMA20	price_vs_EMA10	price_vs_SMA50	BB_position	high_close_ratio	price_vs_SMA20	ROC_10	close_position	..
count	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	..
mean	0.093423	0.081000	0.001699	0.000789	0.004489	0.575570	1.006087	0.001677	0.002913	0.551160	..
std	0.324141	0.225858	0.023310	0.015831	0.043419	0.327422	0.007971	0.027203	0.033115	0.329477	..
min	-0.874166	-0.727213	-0.229793	-0.161694	-0.382447	-0.466826	1.000000	-0.258081	-0.258846	0.000000	..
25%	-0.138439	-0.082610	-0.007518	-0.005678	-0.011477	0.314636	1.001192	-0.009653	-0.012667	0.248221	..
50%	0.126568	0.104804	0.006651	0.003498	0.013900	0.657127	1.003283	0.006772	0.006412	0.587174	..
75%	0.345797	0.251363	0.015598	0.009765	0.030876	0.841644	1.007976	0.017351	0.021179	0.861129	..
max	0.831603	0.733733	0.071909	0.063464	0.125130	1.290744	1.104767	0.101542	0.216384	1.000000	..

8 rows × 21 columns

In [71]:
```
# First Simulated
print(f"==================")
print(f"||  Simulated_1 ||")
print(f"==================")
```

```python
df_first_sim.describe()
```

```
==================
||  Simulated_1 ||
==================
```

Out[71]:

	RSI_7_norm	RSI_14_norm	price_vs_EMA20	price_vs_EMA10	price_vs_SMA50	BB_position	high_close_ratio	price_vs_SMA20	ROC_10	close_position	
count	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6
mean	0.093622	0.081331	0.001619	0.000717	0.004140	0.575013	1.006137	0.001694	0.002886	0.548257	
std	0.325747	0.225181	0.023113	0.015956	0.043419	0.328318	0.008265	0.026759	0.033186	0.328740	
min	-0.874166	-0.727213	-0.229793	-0.161694	-0.382447	-0.466826	1.000000	-0.256975	-0.258846	0.000000	
25%	-0.140249	-0.080137	-0.007194	-0.005696	-0.011338	0.313144	1.001176	-0.009410	-0.012361	0.244674	
50%	0.129987	0.103311	0.006515	0.003446	0.013602	0.655819	1.003256	0.006613	0.006171	0.585869	
75%	0.348661	0.249985	0.015435	0.009775	0.030673	0.845760	1.007802	0.017161	0.021317	0.855655	
max	0.827253	0.668392	0.063312	0.057761	0.122837	1.248946	1.095567	0.099028	0.190525	1.000000	

In [72]:

```python
# After bouds Standartization and cliped by regimes
print(f"==================")
print(f"|| Simulated_2  ||")
print(f"==================")

df_clipped_sim.describe()
```

```
==================
|| Simulated_2  ||
==================
```

Out[72]:

	RSI_7_norm	RSI_14_norm	price_vs_EMA20	price_vs_EMA10	price_vs_SMA50	BB_position	high_close_ratio	price_vs_SMA20	ROC_10	close_position	
count	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6468.000000	6
mean	0.093893	0.081420	0.001627	0.000734	0.004182	0.575405	1.006114	0.001707	0.002900	0.548259	
std	0.324027	0.223730	0.023010	0.015854	0.043253	0.326093	0.008211	0.026631	0.032969	0.328738	
min	-0.858716	-0.692819	-0.223225	-0.154636	-0.350658	-0.389215	1.000000	-0.255754	-0.243906	0.000000	
25%	-0.140249	-0.080137	-0.007194	-0.005696	-0.011338	0.313144	1.001176	-0.009410	-0.012361	0.244674	
50%	0.129987	0.103311	0.006515	0.003446	0.013602	0.655819	1.003256	0.006613	0.006171	0.585869	
75%	0.348661	0.249985	0.015435	0.009775	0.030673	0.845760	1.007802	0.017161	0.021317	0.855655	
max	0.749352	0.568561	0.063312	0.057057	0.121491	1.141591	1.094833	0.099028	0.183629	1.000000	

simulate RETURNS

```
Simulate Returns from factors (β-model) add temporal memory (AR structure)
- regime-conditioned β model
- regime AR(1)
- regime volatility
- fat tails

Regime simulated srtuctural model:
r(t) = mu(r) + b(r)T(F(t) - mu(F,r)) + phi(r) * r(t-1) + eps(t,r)
eps(t,r) ~ t_v(0,sigma(r))
```

In [73]:
```python
# AR() returns on Real world
'''
Calculates the AR(1) autocorrelation coefficient (phi) for returns within each HMM regime separately
'''
def regime_ar1(df_rw, regime):
    df = df_rw.copy(deep=True)

    # Selects only rows belonging to the target regime (0, 1, or 2).
    mask = df["hmm_regime"] == regime

    r_t = df["return"]
    r_lag = r_t.shift(1) # return from previous period
    # Each row now has r_t (today) and r_lag (yesterday).

    # Key filter — both today AND yesterday must be in the same regime.
    # This avoids correlating a return from one regime with a return
    # from a completely different regime context.
    valid = mask & mask.shift(1)

    return np.corrcoef(r_lag[valid], r_t[valid])[0,1]
```

In [74]:
```python
# phi by regime
phi_0 = regime_ar1(vet_features_df, regime=0)   # calm
phi_1 = regime_ar1(vet_features_df, regime=1)   # transition
phi_2 = regime_ar1(vet_features_df, regime=2)   # stress
```

In [75]:
```python
# Real world phi dict
phi_dict_real = {
    0: phi_0,   # calm
    1: phi_1,   # transition
    2: phi_2    # stress
}
```

In [76]:
```
phi_dict_real
```

Out[76]:
```
{0: np.float64(-0.06953743497372676),
 1: np.float64(-0.03864943565307201),
 2: np.float64(-0.14768316509818616)}
```

In [77]:
```
# Estimate on REAL data
def calibrate_regime_model(df_rw, features,
                           return_col="fwd_return",
                           regime_col="hmm_regime"):

    df = df_rw.copy(deep=True)
    betas = {}
    factor_means = {}
    return_means = {}
    #phi_dict = {}
    sigma_dict = {}

    for r in sorted(df[regime_col].unique()):

        sub = df[df[regime_col] == r].copy()

        # -------------------------
        # AR(1)
        # -------------------------
        #phi = sub[return_col].autocorr(lag=1)
        #phi_dict[r] = phi

        # -------------------------
        # Center X and y
        # -------------------------
        X = sub[features]
        y = sub[return_col]

        mu_F = X.mean().values   # numpy.ndarray
        mu_r = y.mean()          # scalar

        X_centered = X - mu_F
        y_centered = y - mu_r

        # -------------------------
        # Regression
        # -------------------------
        reg = LinearRegression()
        reg.fit(X_centered, y_centered)

        beta = reg.coef_

        # -------------------------
```

```
        # Residual volatility
        # -------------------------
        residuals = y_centered - reg.predict(X_centered)
        sigma_r = residuals.std()

        # -------------------------
        # Store
        # -------------------------
        betas[r] = beta
        factor_means[r] = mu_F
        return_means[r] = mu_r
        sigma_dict[r] = sigma_r

    return betas, factor_means, return_means, sigma_dict
```

In [78]:
```
betas, factor_means, return_means, sigma_dict = calibrate_regime_model(vet_features_df, selected_factors_list, return_col="fwd_return", re
```

In [79]:
```
# Smulated Returns Regeme Engine
def simulate_returns_regime_engine(df_sim_sim,
                                   features,
                                   betas,
                                   factor_means,
                                   return_means,
                                   phi_dict,
                                   sigma_dict,
                                   regime_col="hmm_regime",
                                   df_t=8):

    df_sim = df_sim_sim.copy(deep=True)
    sim_r = np.zeros(len(df_sim))

    #---------------------------------#
    # initialise conditional variance #
    h = None
    omega = None
    prev_r = None

    # Starting values (GARCH-lite parameters):
    alpha = 0.02    # shock sensitivity
    beta_g = 0.90   # variance persistance
    #---------------------------------#

    for t in range(len(df_sim)):

        row = df_sim.iloc[t]
        r = int(row[regime_col])

        #---------------------------------#
        # When the regime changes (calm->stress), sigma_r jumps.
```

```
        # We need to reset h at each regime transition to avoid a discontinuity:
        # Re-initialise h when regime changes
        if r != prev_r:
            h = sigma_dict[r]**2
            omega = sigma_dict[r]**2 * (1 - alpha - beta_g) # = sigma_r**2 × 0.05
            prev_r = r
        #---------------------------------#

        beta = betas[r]
        mu_F = factor_means[r]
        mu_r = return_means[r]
        phi = phi_dict[r]
        sigma_r = sigma_dict[r]

        F = row[features].values

        # centered signal
        signal = np.dot(beta, F - mu_F)

        # regime-specific fat-tailed noise. Scaled by today's conditional std
        #eps = np.random.standard_t(df=df_t) * sigma_r
        eps = np.random.standard_t(df=df_t) * np.sqrt(h)

        if t == 0:
            sim_r[t] = mu_r + signal + eps
        else:
            sim_r[t] = mu_r + signal + phi * sim_r[t-1] + eps

        #---------------------------------#
        # Update conditional variance (GARCH-lite)
        h = omega + alpha * eps**2 + beta_g * h
        #---------------------------------#

    df_sim["sim_return"] = sim_r
    return df_sim
```

In [80]:
```
# DF clipped_sim with 'return' simulated
df_clipped_sim_new = simulate_returns_regime_engine(
    df_clipped_sim, selected_factors_list, betas,
    factor_means, return_means, phi_dict_real,
    sigma_dict, regime_col="hmm_regime", df_t=8
)
```

In [81]:
```
#================================ VIZ CLOSE SIM vs REGIME SIM ========================================#
```

In [82]:
```
# Define your date ranges
def viz_sim_close_vs_regime_1500(combined_df):

    date_ranges = [
```

```python
        (0, 1499),
        (1500, 2999),
        (3000, 4499),
        (4500, 5999),
        (6000, 6499)
    ]

    start_date = 0
    end_date = 6468

    # Create subplots
    n_plots = len(date_ranges)
    fig, axes = plt.subplots(n_plots, 1, figsize=(15, 5*n_plots))

    colors = {0: 'blue', 1: 'green', 2: 'red'}

    for idx, (start_date, end_date) in enumerate(date_ranges):
        # Filter data for this date range
        mask = (combined_df.index >= start_date) & (combined_df.index <= end_date)
        df_range = combined_df[mask]

        if len(df_range) > 0:
            ax = axes[idx]

            # Plot each regime
            for regime in df_range['hmm_regime'].unique():
                regime_mask = df_range['hmm_regime'] == regime

                if regime == 0:
                    regime_name = "calm"
                elif regime == 1:
                    regime_name = "transition"
                else:
                    regime_name = "stress"

                ax.scatter(df_range.index[regime_mask],
                           df_range['sim_close'][regime_mask],
                           c=colors.get(regime, 'gray'),
                           label=f'Regime {regime_name}',
                           s=10, alpha=0.7)

            ax.set_title(f'Simulated Period: {start_date} to {end_date}', fontsize=12)
            ax.set_ylabel('Simulated Close Price', fontsize=10)
            ax.grid(True, alpha=0.3)
            ax.tick_params(axis='x', rotation=45)
            ax.legend()

    plt.tight_layout()
    plt.show()
```

In [83]:
```
# Generate realistic price data
regimes_sim = df_clipped_sim_new["hmm_regime"].copy(deep=True)
returns_sim = df_clipped_sim_new["sim_return"].copy(deep=True)
```

In [84]:
```
# Generate realistic price data
start_price = 1000
close_sim = start_price * np.exp(np.cumsum(returns_sim))
# Convert to data frame
regimes_sim_df = pd.DataFrame(regimes_sim)
close_sim_df = pd.DataFrame(close_sim)
```

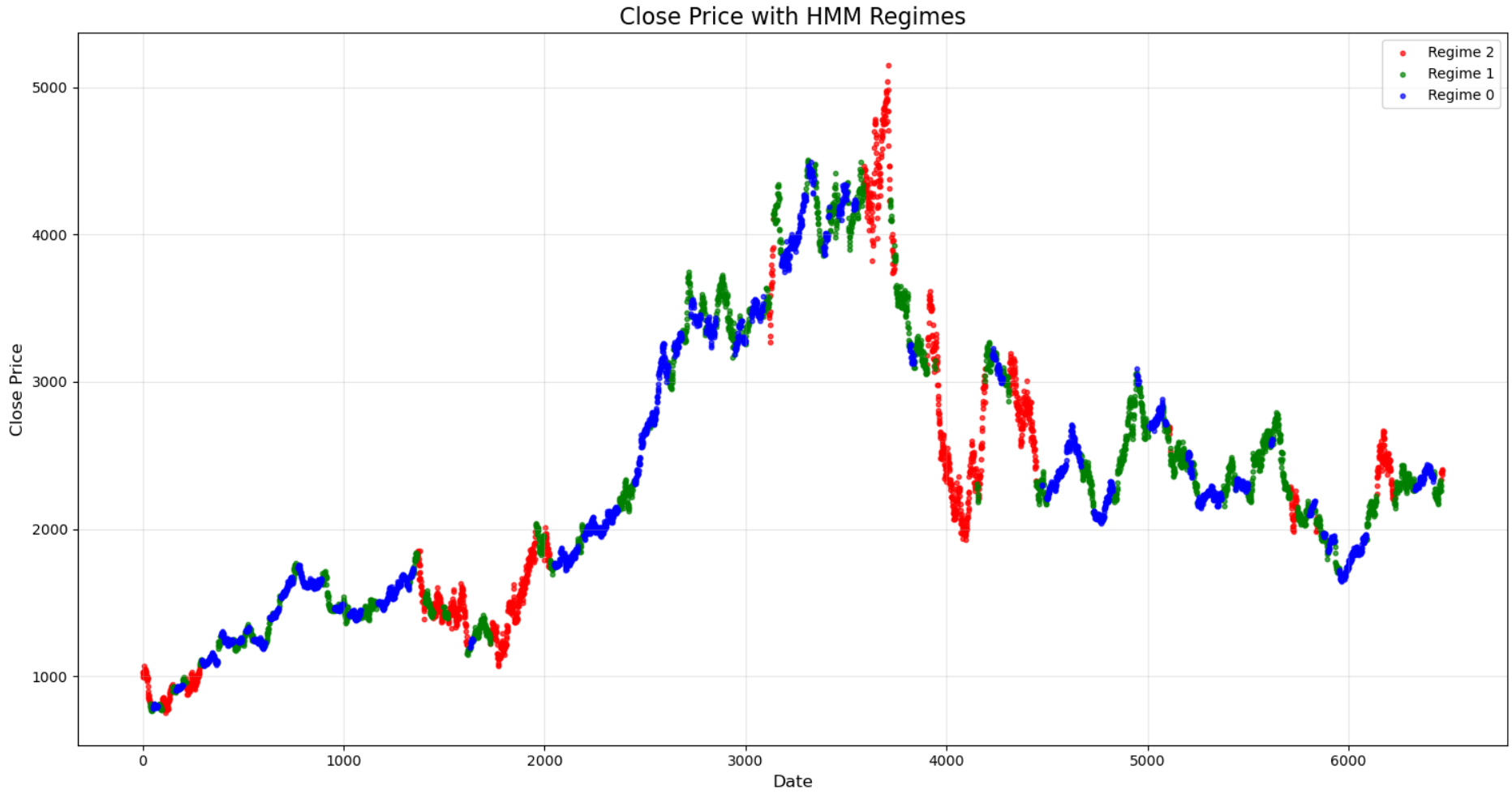
In [85]:
```
# Align the data (make sure dates match)
combined_sim_df = pd.concat([close_sim_df, regimes_sim_df], axis=1).dropna()
combined_sim_df.rename(columns={'sim_return': 'sim_close'}, inplace=True)
```

In [86]:
```
combined_sim_df['sim_close'].min(), combined_sim_df['sim_close'].max()
```

Out[86]:
```
(np.float64(753.4822907689431), np.float64(5149.297862267988))
```

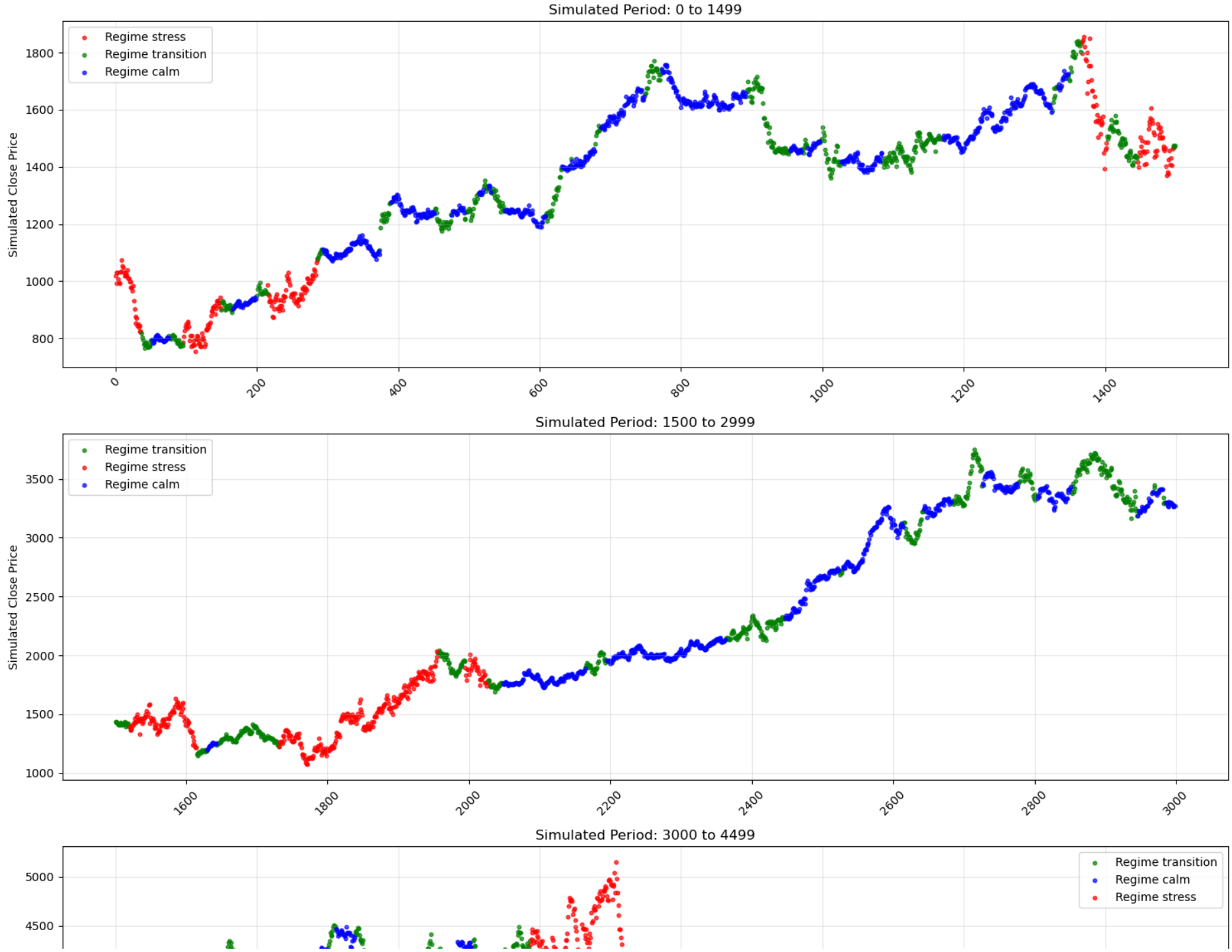
In [87]:
```
def all_data_viz(combined_df):
    # Create the plot
    plt.figure(figsize=(15, 8))

    # Scatter plot with colors based on regime
    colors = {0: 'blue', 1: 'green', 2: 'red'}
    for regime in combined_df['hmm_regime'].unique():
        mask = combined_df['hmm_regime'] == regime
        plt.scatter(combined_df.index[mask],
                    combined_df['sim_close'][mask],
                    c=colors.get(regime, 'gray'),
                    label=f'Regime {regime}',
                    s=10, alpha=0.7)

    plt.title('Close Price with HMM Regimes', fontsize=16)
    plt.xlabel('Date', fontsize=12)
    plt.ylabel('Close Price', fontsize=12)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

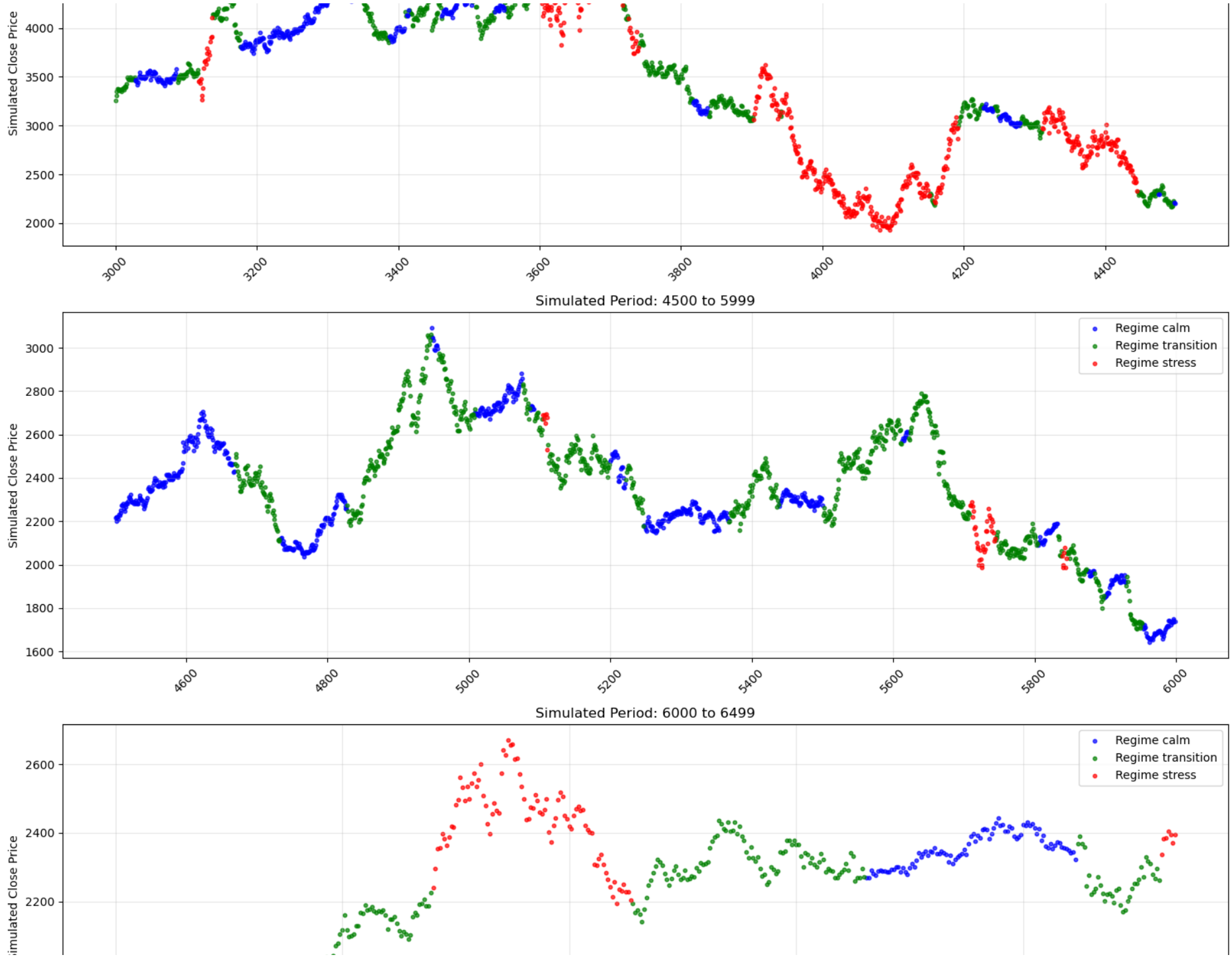
In [88]:
```
all_data_viz(combined_sim_df)
```

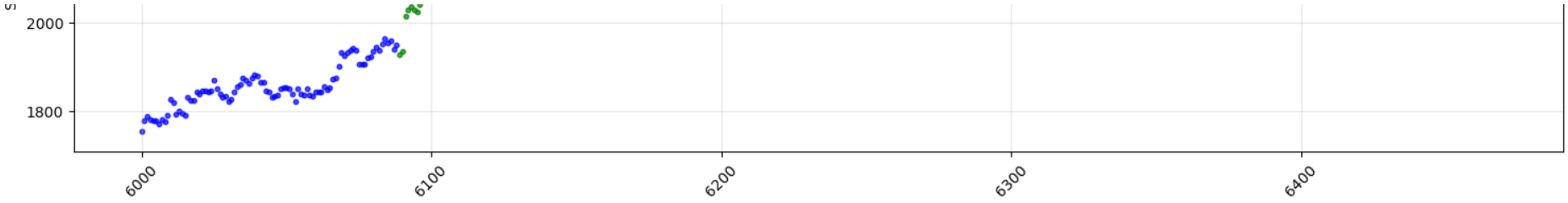


In [ ]:

In [89]:
```
# 0-1500
viz_sim_close_vs_regime_1500(combined_sim_df)
```


Simulated Period: 0 to 1499
Regime stress
Regime transition
Regime calm
Simulated Close Price
1800
1600
1400
1200
1000
800
0
200
400
600
800
1000
1200
1400
Simulated Period: 1500 to 2999
Regime transition
Regime stress
Regime calm
Simulated Close Price
3500
3000
2500
2000
1500
1000
1600
1800
2000
2200
2400
2600
2800
3000
Simulated Period: 3000 to 4499
Regime transition
Regime calm
Regime stress
5000
4500


Simulated Close Price
4000
3500
3000
2500
2000
3000
3200
3400
3600
3800
4000
4200
4400
Simulated Period: 4500 to 5999
Regime calm
Regime transition
Regime stress
Simulated Close Price
3000
2800
2600
2400
2200
2000
1800
1600
4600
4800
5000
5200
5400
5600
5800
6000
Simulated Period: 6000 to 6499
Regime calm
Regime transition
Regime stress
2600
2400
2200



Level Checks Sim vs Real

Destribution

In [90]:
```
# Real data 'fwd_return'
vet_features_df[selected_factors_list + ["hmm_regime"] + ['fwd_return']]["fwd_return"].describe()
```

Out[90]:
```
count    6468.000000
mean        0.000312
std         0.012166
min        -0.119841
25%        -0.004704
50%         0.000645
75%         0.005860
max         0.115800
Name: fwd_return, dtype: float64
```

In [ ]:
```
# df_t=8 GARCH. Simulated data. Clipped. 'fwd_return' Destribution 0.02
df_clipped_sim_new["sim_return"].describe()
```

In [ ]:
```

```

Autocorrelation

In [91]:
```
# Real data
vet_features_df['fwd_return'].autocorr(1)
```

Out[91]:
```
np.float64(-0.10397140272057682)
```

In [92]:
```
# df_t=8 GARCH. Simulated clipped  a=0.02 b=0.90
df_clipped_sim_new['sim_return'].autocorr(1)
```

Out[92]:
```
np.float64(-0.11898191729420561)
```

Regime-wise volatility match

In [93]:
```
# Real Regime Vol
vet_features_df.groupby('hmm_regime')['fwd_return'].std()
```

Out[93]:
```
hmm_regime
0    0.005932
1    0.009972
2    0.021721
Name: fwd_return, dtype: float64
```

In [94]:
```
# df_t=8 GARCH. Simulated Regime vol. Clipped   a=0.02 b=0.90
df_clipped_sim_new.groupby('hmm_regime')['sim_return'].std()
```

Out[94]:
```
hmm_regime
0    0.006999
1    0.012106
2    0.024526
Name: sim_return, dtype: float64
```

Regime Volatility — Correct Shape, Slightly Over-Scaled

Tail quantile comparison

Tail Behavior

In [95]:
```
# Real World
vet_features_df['fwd_return'].quantile([0.01, 0.99])
```

Out[95]:
```
0.01   -0.03419
0.99    0.03353
Name: fwd_return, dtype: float64
```

In [96]:
```
# df_t=8 GARCH. Simulated clipped  a=0.02 b=0.90
df_clipped_sim_new['sim_return'].quantile([0.01, 0.99])
```

Out[96]:
```
0.01   -0.041205
0.99    0.040033
Name: sim_return, dtype: float64
```

Slightly Too Fat (too aggressive in heavy tails)

Factor Distribution

In [97]:
```
# Real world
vet_features_df['RSI_7_norm'].quantile([0.01, 0.99])
```

Out[97]:
```
0.01   -0.641608
0.99    0.715539
Name: RSI_7_norm, dtype: float64
```

In [98]:
```python
# df_t=8 GR Simulated clipped
df_clipped_sim_new['RSI_7_norm'].quantile([0.01, 0.99])
```

Out[98]:
```
0.01   -0.634023
0.99    0.701171
Name: RSI_7_norm, dtype: float64
```

That is extremely close. This confirms:

```
Copula marginals are correct
Clipping barely activates
No distortion after simulation
```

Correlation check

```
If that is near zero -> good (independent world).
If it’s high -> you accidentally leaked information.
```

In [100…
```python
# df_t=8 GARCH a=0.02 b=0.90
sim_return = df_clipped_sim_new['sim_return']
real_return = vet_features_df['fwd_return']

correlation_value = sim_return.corr(real_return.reset_index(drop=True))
print(f"The correlation between df_clipped_sim_new['sim_return'] and vet_features_df['fwd_return'] is: {correlation_value}")

# Calculate the correlation matrix
correlation_matrix = np.corrcoef(sim_return.abs(), real_return.abs())
print(f"The correlation_matrix is:\n {correlation_matrix}")
```

```
The correlation between df_clipped_sim_new['sim_return'] and vet_features_df['fwd_return'] is: -0.012127800443809286
The correlation_matrix is:
 [[ 1.        -0.0465168]
 [-0.0465168  1.       ]]
```

The correlation is near zero -> good, we have independent world

```
~0 means: simulator generates structurally similar but independent universes.
```

Perform the Ljung–Box test

```
Purpose:
Test whether the time series has significant autocorrelation up to a given lag.
```

In [102…

```python
# Real World
acorr_ljungbox(vet_features_df['fwd_return'], lags=[5,10,20])
```

Out[102…

	lb_stat	lb_pvalue
5	78.754034	1.529125e-15
10	107.486566	1.712345e-18
20	171.892935	3.712023e-26

In [ ]:

```python
"""
Very small p-values ->
Strong rejection of “no autocorrelation”.
real returns have a significant sequential structure.
"""
```

In [103…

```python
# df_t=8 GARCH. Simulated clipped a=0.02 b=0.90
acorr_ljungbox(df_clipped_sim_new['sim_return'], lags=[5,10,20])
```

Out[103…

	lb_stat	lb_pvalue
5	98.207570	1.261047e-19
10	121.472125	2.542763e-21
20	150.594961	4.835090e-22

In [ ]:

```python
"""
Also extremely small -> Also strong serial dependence.
Simulator captures:
    - Short memory well
    - Medium memory partially
    - Long memory less strongly
"""
```

Generate Simulated World

In [104…

```python
#==============================================================================
def generate_world(seed,
                   vet_features_df,
```

```
                 selected_factors_list,
                 betas,
                 factor_means,
                 return_means,
                 phi_dict,
                 sigma_dict,
                 trans_mat,
                 models=models,
                 df_t=8,
                 alpha=0.02,
                 beta_g=0.90):
    """
    Generate one complete synthetic world.
    All structure is fixed — only the random seed changes.
    """
    np.random.seed(seed)

    T = len(vet_features_df)

    # 1. Simulate regime path
    sim_regimes = simulate_regimes(T, trans_mat, init_state=0)

    # 2. Count regimes for copula sampling
    count_0 = (sim_regimes['hmm_regime'] == 0).sum()
    count_1 = (sim_regimes['hmm_regime'] == 1).sum()
    count_2 = (sim_regimes['hmm_regime'] == 2).sum()

    # 3. Fit copula models (on real data — fixed)
   # models = fit_copula(vet_features_df, selected_factors_list) - out of the loop now

    # 4. Sample factors per regime
    sim_F0 = sample_copula(models, selected_factors_list, regime=0, n=count_0)
    sim_F1 = sample_copula(models, selected_factors_list, regime=1, n=count_1)
    sim_F2 = sample_copula(models, selected_factors_list, regime=2, n=count_2)

    # 5. Assemble factor DataFrame in regime order
    regime_positions = {0: 0, 1: 0, 2: 0}
    rows_list = []
    for regime in sim_regimes['hmm_regime']:
        regime = int(regime)
        pos = regime_positions[regime]
        if regime == 0:   row = sim_F0.iloc[[pos]]
        elif regime == 1: row = sim_F1.iloc[[pos]]
        else:             row = sim_F2.iloc[[pos]]
        rows_list.append(row)
        regime_positions[regime] += 1

    df_world = pd.concat(rows_list, ignore_index=True)
    df_world['hmm_regime'] = sim_regimes['hmm_regime'].values
```

```
    # 6. Clip to empirical bounds
    df_world = clip_to_empirical_by_regime(
        df_world, vet_features_df, selected_factors_list
    )

    # 7. Simulate returns
    df_world = simulate_returns_regime_engine(
        df_world, selected_factors_list,
        betas, factor_means, return_means,
        phi_dict, sigma_dict,
        regime_col='hmm_regime',
        df_t=df_t
    )

    # 8. Tag world identity
    df_world['world_id'] = seed

    return df_world
#==============================================================================
```

Generate N Simulated Worlds

In [105…

```
# ============= Generate library ==============
N_WORLDS = 50

world_library = []
# Fit once — outside the loop
models = fit_copula(vet_features_df, selected_factors_list)
for seed in range(N_WORLDS):
    world = generate_world(seed, vet_features_df, selected_factors_list,
                           betas, factor_means, return_means,
                           phi_dict_real, sigma_dict, trans_mat, models)
    world_library.append(world)

# Stack all worlds into one DataFrame if needed
all_worlds_df = pd.concat(world_library, ignore_index=True)
print(f"Library: {N_WORLDS} worlds × {len(vet_features_df)} days = {len(all_worlds_df):,} rows")
```

```
Library: 50 worlds × 6468 days = 323,400 rows
```

Destribution

In [107…

```
# Real World
vet_features_df['fwd_return'].describe()
```

Out[107…

```
count    6468.000000
mean        0.000312
std         0.012166
min        -0.119841
25%        -0.004704
50%         0.000645
75%         0.005860
max         0.115800
Name: fwd_return, dtype: float64
```

In [106…

```
# Simulated N Worlds
all_worlds_df['sim_return'].describe()
```

Out[106…

```
count    323400.000000
mean          0.000291
std           0.014365
min          -0.210575
25%          -0.006255
50%           0.000317
75%           0.006855
max           0.192229
Name: sim_return, dtype: float64
```

In [108…

```
# df_t=8 GARCH. Simulated Regime vol. Clipped   a=0.02 b=0.90
all_worlds_df.groupby('hmm_regime')['sim_return'].std()
```

Out[108…

```
hmm_regime
0    0.007100
1    0.011940
2    0.025994
Name: sim_return, dtype: float64
```

In [109…

```
# df_t=8 GARCH. Simulated a=0.02 b=0.90
all_worlds_df['sim_return'].quantile([0.01, 0.99])
```

Out[109…

```
0.01   -0.041919
0.99    0.042505
Name: sim_return, dtype: float64
```

In [ ]:

```
"""
We've built:
Feature Engine → Regime Model → Robust Factor Selection →
Copula Regime Simulation → Beta Model → AR Memory → Vol Calibration

What We Have Achieved Structurally

Simulated world matches real world in:
| Property                | Match?          |
| ----------------------- | ----------------|
```

```
| AR(1) autocorr          | ✅ Almost exact |
| Regime vol ordering     | ✅ Correct      |
| Vol magnitude           | ⚠ Slightly high |
| Tail heaviness          | ⚠ Slightly fat  |
| Factor marginals        | ✅ Very close   |
| Cross-feature structure | ✅ Preserved    |
| Path correlation        | ✅ ~0           |

"""
```

In [114…

```
# Generate realistic price data
def close_and_regime(returns_sim, regimes_sim, start_price=1000):

    close_sim = start_price * np.exp(np.cumsum(returns_sim))
    # Convert to data frame
    regimes_sim_df = pd.DataFrame(regimes_sim)
    close_sim_df = pd.DataFrame(close_sim)

    # Align the data (make sure dates match)
    combined_sim_df = pd.concat([close_sim_df, regimes_sim_df], axis=1).dropna()
    combined_sim_df.rename(columns={'sim_return': 'sim_close'}, inplace=True)

    return combined_sim_df
```

In [ ]:

```
n_combined_sim_df = close_and_regime(all_worlds_df['sim_return'], all_worlds_df['hmm_regime'], start_price=1000)
```

In [130…

```
n_combined_sim_df
```

Out[130…

	sim_close	hmm_regime
0	1.006066e+03	0
1	1.013694e+03	0
2	1.008207e+03	0
3	1.007398e+03	0
4	1.003698e+03	0
...	...	...
323395	6.992925e+43	0
323396	6.999860e+43	0
323397	7.039419e+43	0
323398	7.010112e+43	0
323399	6.988588e+43	0

323400 rows × 2 columns

In [129…

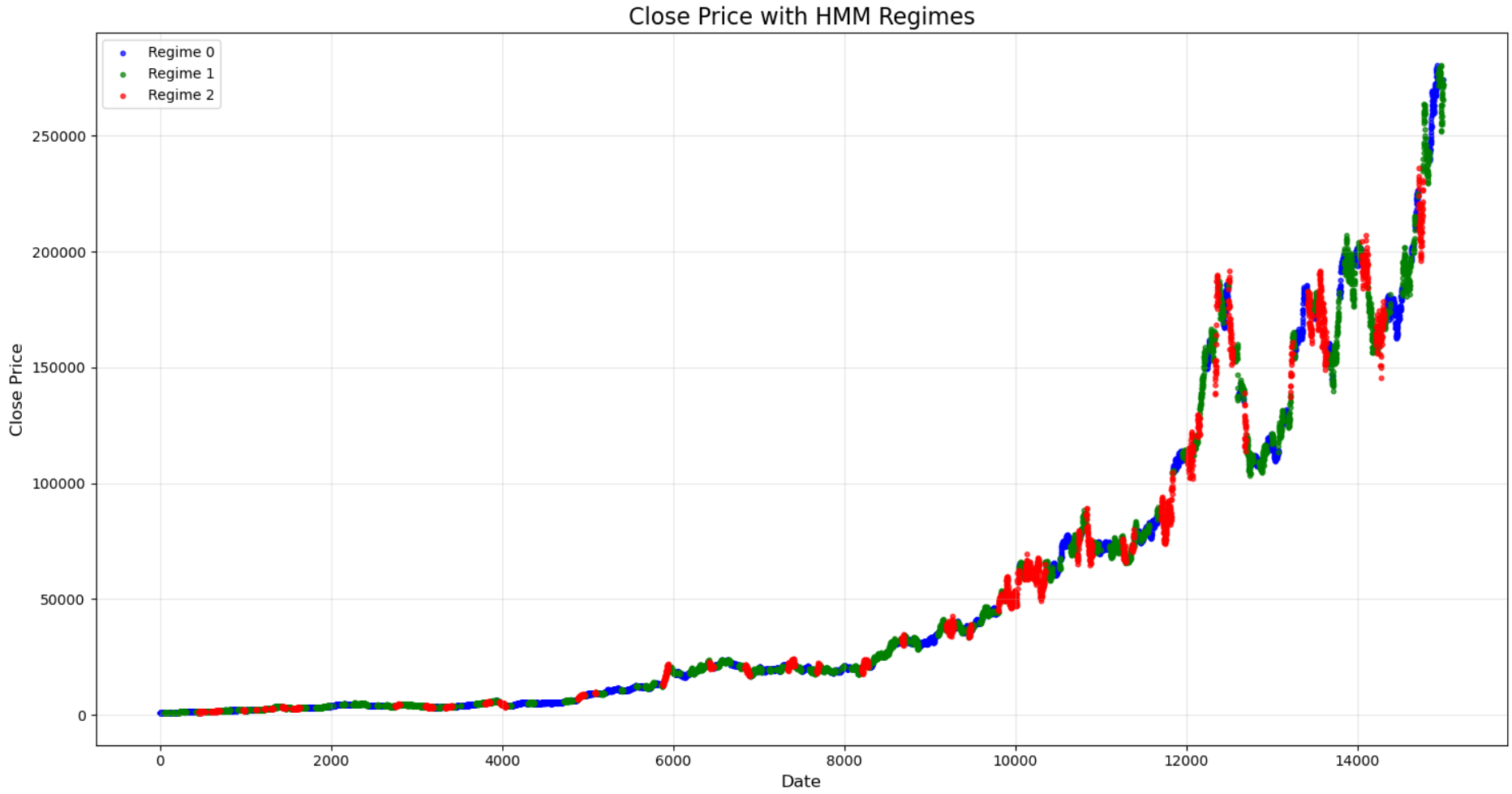
```
n_combined_sim_df['hmm_regime'].value_counts()
```

Out[129…

```
hmm_regime
1    138358
0    124782
2     60260
Name: count, dtype: int64
```

In [146…

```
all_data_viz(n_combined_sim_df.iloc[: 15000])
```


Close Price with HMM Regimes
Regime 0
Regime 1
Regime 2
Close Price
Date
0
50000
100000
150000
200000
250000
0
2000
4000
6000
8000
10000
12000
14000

In [ ]: